



Lecture 5: Handling User Events

INTERVIEW QUESTION

[> CORE](#)

How do you handle user events like clicks in Svelte?

You want a counter that goes up every time the user clicks a button. Where does the click handler go, and how does it connect to your state?

Interactive web applications rely on responding to user actions: clicking buttons, submitting forms, moving the mouse, or pressing keys.

The Problem with Manual Event Listeners

Imagine building a checkout page where a user clicks a "Buy Now" button to complete their purchase.

In vanilla JavaScript, to make that button do something, you have to query the DOM: `document.getElementById('buy-btn').addEventListener('click', buyItems)`. This is fragile. If you accidentally rename the ID in your HTML but forget to update the JavaScript, the connection is severed. Your user clicks "Buy Now", nothing happens, and you lose the sale. Furthermore, if you dynamically create and destroy buttons, you have to manually detach those event listeners to prevent catastrophic memory leaks. The sheer disconnect between the HTML element and its JavaScript behavior creates a massive surface area for bugs.

We need a way to bind the behavior *directly* to the HTML element where it is defined, so they are impossible to separate.

Event Attributes

To solve this, Svelte 5 allows you to pass JavaScript functions directly into standard HTML attributes like `onclick`, `oninput`, or `onkeydown`. This is known as using **event attributes**. By wiring the event listener directly in the template, Svelte guarantees it is safely attached when the element is created and perfectly cleaned up when the element is destroyed.

KEY TAKEAWAY

In Svelte 5, event listeners are just regular properties. They don't require special framework syntax.

Wiring a Click Handler

To handle a click event, you simply define a function in your `<script>` and assign it to the `onclick` attribute of your HTML element.



The screenshot shows a code editor window titled "Checkout.svelte". The code is as follows:

```
1 <script>
2   let items = $state(0);
3
4   function buyItem() {
5     items++;
6   }
7 </script>
8
9 <button onclick={buyItem}>
10   Buy Item ({items})
11 </button>
```

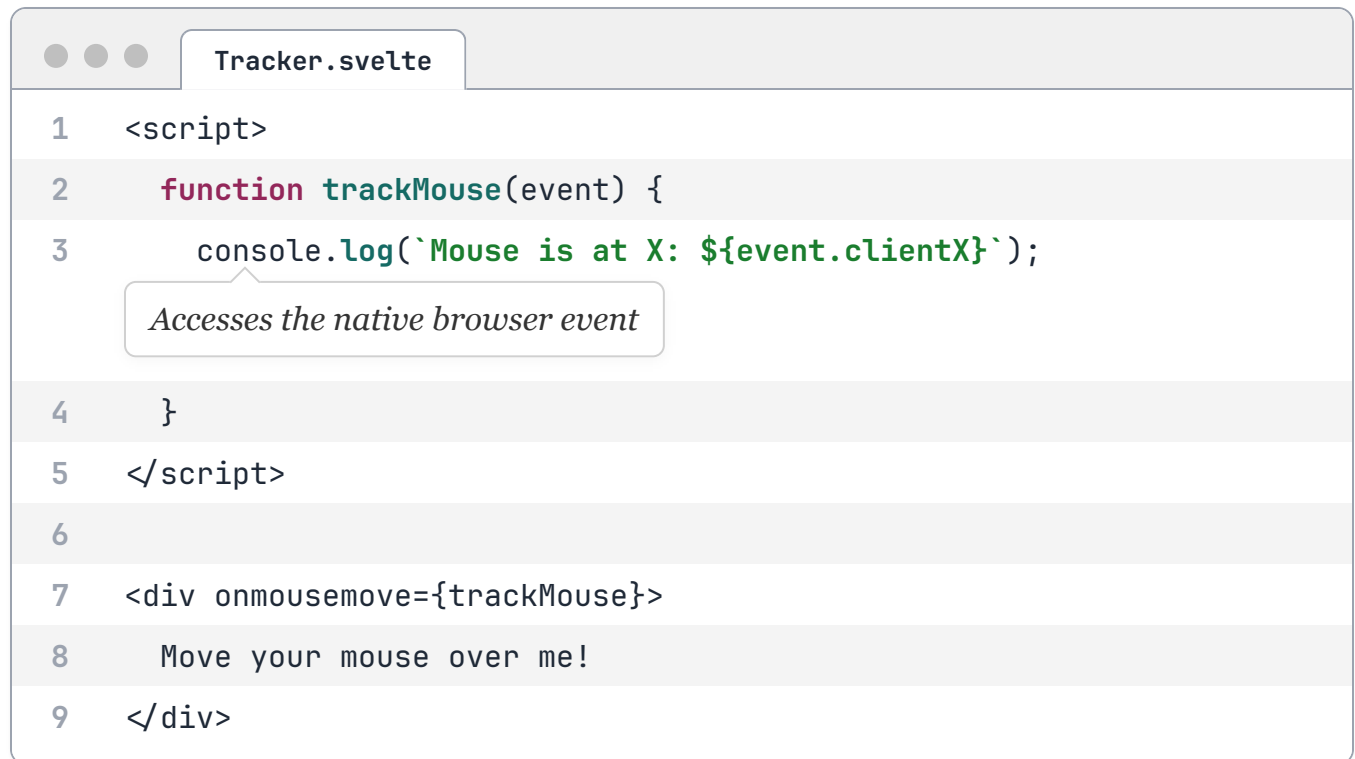
There are two callout boxes in the image:

- A callout box pointing to line 5 (`items++;`) with the text: *Updates the reactive state*
- A callout box pointing to line 9 (`onclick={buyItem}`) with the text: *The onclick attribute wires the button directly to the function*

Notice the syntax: `onclick={buyItem}`. We pass the function reference directly inside the curly braces. We do *not* invoke the function like `onclick={buyItem()}` —doing so would run the function immediately when the page loads, instead of waiting for the user to click.

Accessing the Event Object

Sometimes you need details about the user's interaction, such as where they clicked on the screen or what key they pressed. When the browser triggers an event, it automatically passes an `event` object to your handler.



```
Tracker.svelte
1  <script>
2    function trackMouse(event) {
3      console.log(`Mouse is at X: ${event.clientX}`);
4    }
5  </script>
6
7  <div onmousemove={trackMouse}>
8    Move your mouse over me!
9  </div>
```

A callout box points to the `event` parameter in the `trackMouse` function, containing the text: *Accesses the native browser event*

INTERVIEW TIP

To impress the interviewer: Mention that Svelte 5 changed event handling from Svelte 4's custom directive syntax (`on:click={handler}`) to standard HTML attributes (`onclick={handler}`). This change aligns Svelte closer to native browser standards and allows for much easier passing (spreading) of event handlers as props.

Summary

You handle user events in Svelte by assigning functions directly to HTML attributes like `onclick` . The function lives in your `<script>` block and can freely modify your `$state` variables. Svelte automatically manages attaching the native event listeners, ensuring your code remains tightly coupled to the UI without the fragility of manual DOM queries.

	MANUAL (Vanilla JS)	SVELTE 5 (<code>onclick</code> attribute)
Binding Method	<code>element.addEventListener('click', fn)</code>	<code>onclick={fn}</code> in template
Element Targeting	Fragile <code>id</code> or <code>class</code> queries	Bound directly to the HTML tag
Memory Management	Must manually <code>removeEventListener</code>	Automatically cleaned up by compiler